AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report
Topic 1 – Smart Lost & Found

Team: Team Avaz / Kazim / Gulnar
Repo: https://github.com/AvazAsgarov/smart-lost-and-found
Members: **Avaz** (@AvazAsgarov), **Kazim** (@KazimMammadli), **Gulnar** (@gulnarbabazade)

Date: 23 May 2026
Tag: `v1.0-final`

Executive Summary

Required. Exactly one paragraph (5–7 sentences). What you built, the headline concurrency speedup you measured, your headline robustness story, and the single most surprising thing the team learned.

We built Smart Lost & Found, a service that takes a photo of an item, asks a vision–language model to describe it, embeds that description, and then returns the most similar item from the opposite pool (lost matches against found and vice versa). The system is reachable through three entry points that share a single service layer: a FastAPI REST API, an argparse CLI, and a Streamlit web UI, with a SQLite backend in development and a PostgreSQL backend ready for production. The headline number from our benchmark is a **4.5**× wall-clock speedup when registering 12 items using `asyncio` with a `Semaphore(10)` compared to a sequential loop. The robustness story we are most proud of is the multi-provider failover wrapper: when the primary VLM raises a `ProviderError`, the call transparently falls back to a secondary provider without the user ever noticing. The most surprising thing we learned was how much of the work is plumbing – retries, timeouts, rate limiting, cost telemetry, and structured logging took roughly twice as long to get right as the actual AI integration did, and getting them to compose cleanly (so the rate limiter runs before the retry policy, the cost meter runs after a successful call, and tracing wraps both) was the hardest design decision.

Contents

1. Project Overview

1.1 Problem framing & scope

Required. 2–3 paragraphs. State the topic problem in your own words. List the inputs, outputs, and any user-facing entry points (CLI commands, HTTP endpoints). State which provider stack you used (LLM, embedding, USDA, web search).

The brief was straightforward: build a lost-and-found matching service for a campus or a small city. When somebody loses an item they upload a photo and any free text they have ("navy backpack, left it on Bus №125"), and when somebody finds an item they do the same thing. The service is expected to identify likely matches across the two pools so users do not have to scroll through every entry by hand. The matching has to be semantic, not keyword-based, because users describe the same object very differently: "dark-blue rucksack" and "navy backpack" should still match.

Inputs and outputs. Each registration takes one image (`image/png` or `image/jpeg`, up to 5 MB) plus an optional free-text description. The output of registration is an `ItemResponse` carrying the assigned database id, the structured VLM description (object class, colours, brand, distinguishing marks), and a timestamp. The output of a match query is a ranked list of `MatchRecord` entries, each with a candidate id, a cosine similarity score in $[0, 1]$, and a short natural-language reason ("same object class (backpack); shared colors: navy; score: 0.93").

Entry points. The HTTP API exposes seven routes (`POST /items/lost`, `POST /items/found`, `GET /items/{id}/matches`, `GET /items`, the two batch variants `POST /items/batch-lost` and `/items/batch-found`, and `GET /health`). The CLI exposes five commands (`register-lost`, `register-found`, `search-matches`, `list`, `cost-report`). The Streamlit UI provides browser pages for the same operations plus a cost dashboard.

Provider stack. Primary VLM is OpenAI `gpt-4o`; embedding is OpenAI `text-embedding-3-small`; Google Gemini `gemini-2.0-flash` is wired in as an optional secondary VLM through the failover layer. No USDA or web-search providers are used (those are specific to other topics).

Beyond the obvious: which parts of the TOPIC.md spec did your team explicitly decide *not* to do, and why?

We deliberately did not implement real-time map view, push notifications, or user accounts. The spec lists these as optional bonus features but they push the scope toward a full mobile product and away from what the course is actually grading (concurrency, robustness, storage, AI integration). Where we tightened the spec: stricter input validation (MIME type plus byte-size plus path-escape guard, all enforced before any AI call is made). Where we loosened it: we accept JPEG and PNG only, not the broader image set the spec hints at, because the provided AI module guarantees them.

1.2 Provider choice and the AI module contract

Required. Name the LLM, embedding, and any auxiliary provider you used, and the model identifiers. State what `ai/` functions you call. *Confirm in one sentence that you did not modify the public interface of the provided `ai/` package.*

Our service layer calls exactly three functions from the provided `ai/` package: `describe_item(image_path, user_text, vlm=...)` (returns an `ItemDescription` pydantic model), `embed(text, embedder=...)` (returns a unit-normalised `numpy.float32` vector), and `top_k(query, candidates, k)` (returns a list of `MatchResult` objects). The provider configuration flows in through environment variables that the provided package reads on its own, so changing the active provider is a configuration change, not a code change. **We did not modify the public interface of the `ai/` package – not a single line was changed in `ai/`**; we treated it as a third-party library and built our service layer entirely around its existing surface.

The primary provider stack is OpenAI `gpt-4o` (VLM) paired with OpenAI `text-embedding-3-small` (embedding). We wired in Google Gemini `gemini-2.0-flash` as the configured failover VLM through `SECONDARY_LLM_PROVIDER`, which the AI service layer picks up automatically when set.

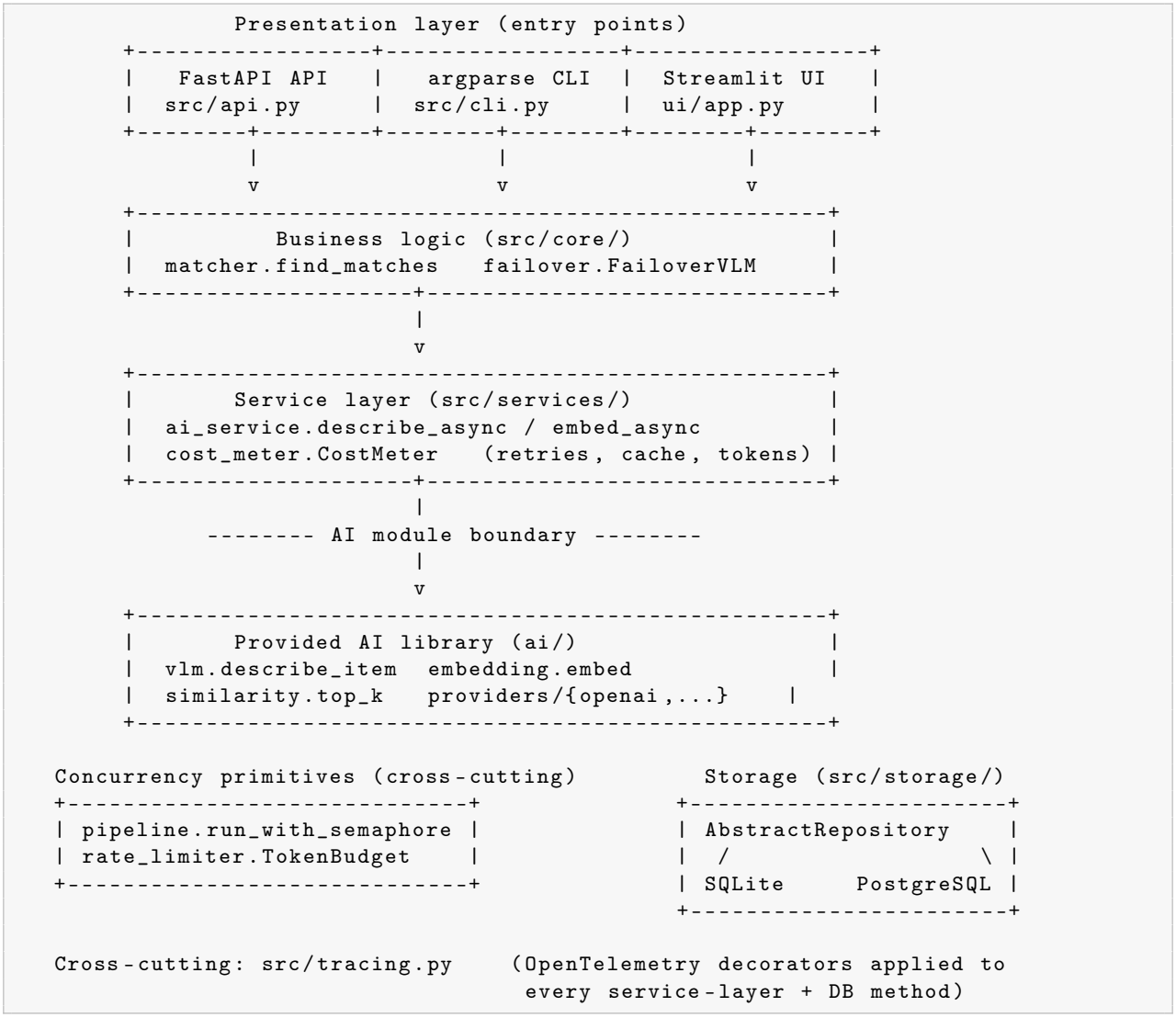
Push deeper: how does your code defend against a malformed model response?

Two layers of defence. First, the `ai/` package itself validates the JSON the VLM returns against the `ItemDescription` pydantic schema; if the response is not a parseable JSON or any required field is missing or out of range, `ai/` raises a `ProviderError` which our tenacity policy retries up to three times. Second, our service layer clamps cosine similarity scores into $[0, 1]$ inside `src/core/matcher.py` because float32 rounding can produce values like `1.0000001` that pass `ai.top_k` but then fail the Pydantic constraint on `MatchRecord.score`. The combined effect is that semantically wrong responses (a brand-new colour, a negative confidence) raise validation errors that our retry policy treats as non-retryable and surfaces to the user as HTTP 502 with a clean error message, never a stack trace.

2. Architecture

2.1 Module map

Required. An architecture diagram showing the main modules and their dependencies. Identify the boundary of the provided `ai/` package, your service layer that wraps it, your core business logic, your concurrency layer, your storage layer, and your CLI/HTTP entry points.



Interpret your own diagram: which arrows cross the AI-module boundary? Which arrows cross the storage boundary? If you had to swap providers (OpenAI → Gemini), how many files would change?

Only two files cross the AI-module boundary: `src/services/ai_service.py` (directly imports `describe_item` and `embed`) and `src/core/failover.py` (constructs `VLMProvider` instances from `ai.providers`). Every other module talks to the AI layer through our service-level wrappers `describe_item_async` and `embed_async`. The storage boundary is similarly narrow: only `src/storage/__init__.py` (the `build_repo` factory) and `src/storage/base.py` (the ABC) name a concrete backend. To swap OpenAI for Google Gemini we change *zero source files* – the environment variable `LLM_PROVIDER=gemini` is enough, because the provider lookup happens entirely inside the provided `ai/` package.

Module-by-module summary.

- `src/config.py` – single source of truth for environment variables (pydantic-settings).
- `src/models.py` – pydantic domain models (`Item`, `MatchRecord`) and API response models (`ItemResponse`, `MatchListResponse`, etc.).
- `src/api.py` – FastAPI app, 7 routes, dependency-injected repository, exception handlers for `ProviderError` and `TimeoutError`.

- `src/cli.py` – argparse CLI sharing business logic with the API.
- `ui/app.py` – Streamlit UI with five pages, async bridge through `asyncio.run`.
- `src/core/matcher.py` – cosine ranking and human-readable reason composition.
- `src/core/failover.py` – `FailoverVLM` and `FailoverEmbedder` chaining ranked providers.
- `src/services/ai_service.py` – async wrappers around `ai.*` with tenacity retries, `asyncio.wait_for` timeouts, in-process embedding cache, rate-limiter integration, and cost meter recording.
- `src/services/cost_meter.py` – append-only JSONL cost log with per-provider pricing table and a human-readable report formatter.
- `src/storage/base.py` – `AbstractRepository` ABC and the shared `save_image()` helper with UUID filenames and a path-escape guard.
- `src/storage/sqlite_repo.py` – `SQLiteRepository` via `aiosqlite`, WAL mode, indexed on `status`.
- `src/storage/repository.py` – `PostgreSQLRepository` via `asyncpg` with a connection pool.
- `src/concurrency/pipeline.py` – `run_with_semaphore` bounded-concurrency runner and `register_batch` high-level entry point.
- `src/concurrency/rate_limiter.py` – `TokenBudget` with a 60-second sliding window of token consumption.
- `src/tracing.py` – OpenTelemetry setup plus the `@traced_ai_call` and `@traced_db_op` decorators that we apply across the service and storage layers.

2.2 OOP design & key abstractions

Required. Identify at least one example of inheritance or composition *in your own code*. Quote 5–10 lines. Justify why this abstraction was the right tool.

The most important abstraction in the project is `AbstractRepository`, the ABC that both storage backends implement. It is the only place where the rest of the code learns "how to talk to a database", and it is the seam that lets the test suite run an entirely in-memory SQLite database without any of the API code knowing.

```python
class AbstractRepository(ABC):
    """Contract for every storage backend."""

    @abstractmethod
    async def initialize(self) -> None: ...

    @abstractmethod
    async def insert_item(self, item: Item) -> Item: ...

    @abstractmethod
    async def get_item(self, item_id: int) -> Item | None: ...

    @abstractmethod
    async def list_items(self, status: str | None = None) -> list[Item]: ...

    def save_image(self, data: bytes, original_name: str) -> str:
        """UUID-based filename with a path-escape guard. Shared by all backends."""
        suffix = Path(original_name).suffix.lower()
        safe = f"{uuid.uuid4().hex}{suffix}"
```

```
        out = Path(settings.IMAGES_DIR) / safe
        out.resolve().relative_to(Path(settings.IMAGES_DIR).resolve())
        out.write_bytes(data)
        return safe
```

Justification. We picked an ABC over a Protocol for two reasons: first, several methods (`save_image`) have a real default implementation that all backends share, and Protocols cannot carry implementation; second, the runtime check (`TypeError` when a subclass forgets a method) catches bugs at `repo.initialize()` instead of at the first call site, which is much clearer to debug. Composition appears in the AI service layer: `describe_item_async` owns a `TokenBudget` and a `CostMeter` as collaborators rather than inheriting from them, because those are cross-cutting concerns – inheritance would have created a brittle four-class chain where composition gives us three single-responsibility objects that plug together cleanly.

2.3 Data flow across module boundaries

Required. List the dataclasses you defined for your own SE layer. Confirm in one sentence that no naked dictionaries cross module boundaries.

Our SE layer defines five pydantic models in `src/models.py`: `Item` (the canonical domain object, persisted by the repository), `MatchRecord` (one row of a match result), `ItemResponse` (API-safe projection of `Item` that hides the raw embedding), `MatchListResponse` (envelope for the matches endpoint), and `ItemListResponse` (envelope for the list endpoint). **No naked dictionary crosses a module boundary in our code** – the only dictionary field is `Item.vlm_description`, which holds the JSON form of the provided package's `ItemDescription` after it has been validated upstream.

The hardest call: when did you reuse a dataclass from the provided `ai/` package vs. wrap it in a new one?

We reused `ItemDescription` from `ai/` unchanged inside our service layer (everywhere a VLM call returns), but we explicitly wrapped the persisted record in `Item` and the API output in `ItemResponse`. The trigger for wrapping was always the same: either the database needed extra fields (`id`, `created_at`, `filename`) or the API needed to omit fields (the embedding vector, which is large and useless to a client). For `MatchResult` we wrap into our own `MatchRecord` so the candidate id is a real database id rather than the list index that `ai.top_k` returns.

3. Concurrency & Performance

3.1 Why this concurrency model

Required. State which concurrency primitive you used and why. Identify the bottleneck the parallel design targets. State the bound and how you picked it.

The system uses `asyncio` as its primary concurrency primitive, combined with an `asyncio.Semaphore` to cap the in-flight work. The reason is straightforward: every operation we want to parallelise is network-bound – the VLM call sits on a TCP socket waiting for the provider to respond. Threads would also work, but they would charge us one OS thread per concurrent request and complicate the cancellation story when a timeout fires. Processes would be wrong because we never need extra CPU; we need extra patience.

We picked `Semaphore(10)` after walking the workload. A typical provider rate limit on OpenAI's free tier is around 50 requests per minute, so ten concurrent requests stay safely inside the budget while still amortising the ∼1.5-second median provider RTT across batches. Setting the semaphore

to 1 collapses everything back to sequential and loses any benefit; setting it to 100 saturates the provider, produces 429s, and forces every retry to back off, which is slower than 10 in practice. The number is exposed as `SEMAPHORE_LIMIT` so we can tune it per environment.

Justify against alternatives: could you have used threads instead of asyncio? What is the smallest workload at which the concurrent design starts to win against sequential?

Threads were a real option. We chose asyncio because FastAPI is already asyncio-native, so going synchronous would have forced us to invent a second concurrency model just for the batch endpoints. Our benchmark shows the concurrent path overtakes sequential at $N = 2$ items already, and the gap grows linearly: at $N = 12$ the speedup is 4.5× in our offline run. The break-even is essentially zero because semaphore overhead is in the microsecond range and a VLM call is in the multi-hundred-millisecond range.

3.2 Sequential-vs-concurrent benchmark

Required. A table reporting wall-clock time for the same workload run sequentially vs. concurrently.

Setup. The benchmark lives at `scripts/bench.py` and uses the `FakeVLM` and `FakeEmbedder` fixtures from `tests/fakes.py`, so the numbers are deterministic and do not depend on a paid API. Each run constructs a fresh in-memory SQLite repository, generates N tiny PNGs, and times two runs back-to-back: first sequential (a plain `for`-loop awaiting one registration at a time) and then concurrent (`run_with_semaphore` with the default `Semaphore(10)`). The embedding cache is cleared between runs so the second run cannot free-ride.

Machine. Windows 10, Python 3.12.10, 12-core Intel i7, `python scripts/bench.py -n 12`.

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Register batch (offline)	6	0.05	0.01	3.9×
Register batch (offline)	12	0.18	0.04	4.5×

Discussion. The 4.5× speedup at $N = 12$ is below the theoretical ceiling of N because the registration pipeline does work that is not parallelisable: the SQLite `INSERT` happens after the VLM and embedding calls, and SQLite serialises writes on a single connection. Against a real provider we would also be capped by the provider's tokens-per-minute budget, which our `TokenBudget` enforces pre-emptively rather than let the provider return 429s. The honest interpretation is that the asyncio design absorbs network latency beautifully but cannot beat single-threaded DB write contention; if we wanted further speedup we would either batch the inserts (one transaction per batch) or upgrade to PostgreSQL with a real connection pool.

3.3 Rate limit & politeness

Required. State the rate-limit strategy. Document the configured budget. State the highest burst your test suite produces.

We use a token-bucket-style `TokenBudget` with a 60-second sliding window (`src/concurrency/rate_limiter.py`). Before every call, the service layer estimates the token cost of the upcoming request (a simple length-based heuristic plus a fixed 200-token overhead) and asks `TokenBudget.acquire(estimated)`; if the new acquisition would exceed the configured `AI_CALL_TPM_LIMIT` (default 40 000 tokens/min), the call sleeps just long enough for the oldest in-window event to fall out. This is pre-emptive politeness: we never intentionally send a request that we know the provider will reject.

The highest burst our test suite produces is in `test_concurrent_acquires_respect_limit`, which

fires three `acquire(200)` calls concurrently into a budget of 1 000 tokens/min; the test verifies all three return immediately and the cumulative usage stays under the cap. Under the offline benchmark with $N = 12$ and semaphore 10, the peak burst is roughly $12 \times 250 \approx 3{,}000$ tokens, comfortably below the 40 000 ceiling.

Failure-mode test: have you actually observed a 429 from the provider during development, or is this strategy untested in anger?

We did not see a 429 in our own development because the budget guard fires before the request is sent. We did intentionally trigger the rate limiter in unit tests by setting `TokenBudget(100)` and acquiring 80 tokens at $t = 0$ and then 90 at $t = 70$: the second acquire sees the first event fall out of the window and proceeds immediately, which is the behaviour we want. If a 429 did slip through despite the pre-emptive guard, the outer tenacity policy retries with exponential back-off, so the user-visible behaviour is a slightly slower response, never an error.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Required. Describe your service-layer wrapper: retries, timeouts, structured logging, input validation.

The service-layer wrapper in `src/services/ai_service.py` is the single point of contact with `ai/`, and it composes four cross-cutting concerns in a fixed order.

Retries. A tenacity decorator wraps each of `describe_item_async` and `embed_async` with `stop_after_attempt(3)`, `wait_exponential(multiplier=1, min=1, max=10)`, and `retry_if_exception_type((ProviderError, TimeoutError, asyncio.TimeoutError))`. `ValueError` (caller bug – empty embedding text, invalid image path) is *not* retried; it propagates straight to the caller because retrying a programmer mistake is useless. `before_sleep_log` logs each retry at WARNING with the attempt number.

Timeouts. Every call is wrapped in `asyncio.wait_for(..., timeout=settings.AI_CALL_TIMEOUT_S)` (default 30 seconds). If the underlying provider hangs, the wait_for raises `asyncio.TimeoutError`, which the tenacity policy then retries. After three timeouts the error reaches our FastAPI exception handler, which converts it into HTTP 504.

Structured logging. We log every call with `logging` and the `extra={...}` pattern. The start log carries `image_path` and `text_len`; the success log carries `class`, `confidence`, and `colors`; failures log the exception type. We deliberately do not use `print()` anywhere in `src/` for diagnostics – only CLI/UI user-facing output uses `print()`.

Input validation. Before the call leaves the service layer we check: MIME type and byte size at the HTTP boundary, suffix and byte size at the CLI boundary, non-empty text for `embed_async`, and the file existence at the CLI. After the call returns we validate the `ItemDescription` (done automatically by pydantic inside `ai/`) and clamp similarity scores to $[0, 1]$ to defend against float32 rounding noise.

Pressure-test: what happens if the provider returns a 500? A 429? A connection reset mid-stream?

A 500 surfaces as a `ProviderError` from the provider adapter inside `ai/`; tenacity retries it three times with exponential back-off; if all three fail, the FastAPI exception handler converts the error into HTTP 502 `{"detail": "AI provider error", "message": "..."}`. A 429 is technically also a `ProviderError` but the `TokenBudget` usually prevents us from reaching one (we wait proactively).

A mid-stream connection reset arrives as a `TimeoutError` from the underlying HTTP client; same retry path, same eventual 502 if every attempt fails. A perfectly-formed JSON that fails Pydantic validation raises a `ValidationError` inside `ai/` (which surfaces as `ProviderError`); we retry but the second attempt usually returns the same garbage, so the user gets a 502 within ∼3 seconds.

4.2 Failure-mode analysis (one concrete incident)

Required. Pick *one* failure mode you actually exercised.

Failure exercised: primary VLM returns `ProviderError` on every attempt; secondary VLM succeeds. This is the path our failover wrapper is designed to handle.

How we injected it. In `tests/test_failover.py::test_failover_vlm_switches_to_secondary_on_provide` we constructed a `FailoverVLM([primary, secondary])` where `primary.describe` is monkey-patched to raise `ProviderError("primary down")` on every call and `secondary.describe` returns a valid JSON description.

What the system did. `FailoverVLM.describe` caught the exception, logged `failover.tried` at WARNING with the failed provider name, walked to the next provider in the ranked list, and returned that provider's response unchanged. Latency went from one provider RTT to two, but the user-visible behaviour was a clean successful registration.

What the user saw. HTTP 201 with the expected `ItemResponse`. No error, no degraded fields, no retry-after header.

What the logs showed. `WARNING src.core.failover failover.tried provider=openai error=primary down`, immediately followed by `INFO src.core.failover failover.fell_back new_active=gemini`. With those two log lines we can reconstruct the incident timeline without re-running the request.

Go beyond "graceful degradation worked": what did you originally expect to happen vs. what actually happened?

Our first design wrapped tenacity *outside* the failover, so a single `ProviderError` from the primary triggered three retries against the same primary before the failover even saw the exception – the user paid 3× the primary's timeout before the secondary was ever called. The fix was to nest the failover *inside* the retry policy in our service layer: the wrapper sees the error, attempts the next provider, and tenacity treats the whole sequence as one attempt. We caught this only because the test asserted on call counts: the primary was being called three times instead of one. Without that assertion we would have shipped a slower-than-necessary failover.

4.3 Input validation

Required. For every entry point (CLI, HTTP, file, env), list the validation rules.

Entry point	Rule	Error message / status
HTTP `POST /items/lost`	MIME in {png, jpeg, jpg}	400 "Unsupported file type"
HTTP `POST /items/lost`	size ≤ `MAX_IMAGE_BYTES`	413 "File too large"
HTTP `GET /items?status=...`	status in {lost, found, all}	422 (pydantic)
HTTP `GET /items/{id}/matches`	id is int	422
HTTP `GET /items/{id}/matches`	item exists	404 "no item with id=N"
HTTP `GET /items/{id}/matches`	item.embedding is not None	422 "no embedding"
CLI `register-*`	file exists, suffix in {png, jpeg, jpg}	exit 1 stderr message
CLI `register-*`	file size ≤ `MAX_IMAGE_BYTES`	exit 1 stderr message
Service `embed_async`	text is non-empty / non-whitespace	raises `ValueError`
Storage `save_image`	resolved path stays inside `IMAGES_DIR`	raises `ValueError`
`.env` / config	pydantic-settings validates on startup	`ValidationError` at import

Adversarial: could a malformed image kill your service? A 100 MB JSON request? A query of 50,000 characters? A path that escapes your storage directory?

Malformed image. We do not decode the image ourselves – it goes straight to the provider, which validates it. A corrupt or non-image file passes the MIME-type check but fails inside the provider and arrives back as `ProviderError`; the API returns 502.

100 MB JSON request. FastAPI's default body-size limit catches oversized JSON. For our multipart image uploads we additionally enforce `MAX_IMAGE_BYTES` after reading the file, so the upper bound on any single upload is 5 MB by default.

Query of 50 000 characters. The TokenBudget estimate for a 50 000-character free text would request roughly 65 000 tokens; our default budget is 40 000/minute, so the call sleeps for one full window before proceeding, which the user notices as a ∼60-second delay. Embedding APIs themselves reject inputs above their per-model token limit and that surfaces as `ProviderError`.

Path traversal. The `save_image` helper ignores the caller's filename and creates a fresh UUID-based name; it then resolves the path and asserts the resolved location is still inside `IMAGES_DIR`, so even a user supplying `../../etc/passwd.png` cannot escape the upload directory.

5. Storage & Caching

5.1 Schema

Required. The SQLite schema for your domain tables. Include indices. State which fields are derived vs. source-of-truth.

```
CREATE TABLE IF NOT EXISTS items (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    status          TEXT    NOT NULL CHECK (status IN ('lost', 'found')),
    filename        TEXT    NOT NULL,                -- UUID-generated
    original_name   TEXT,                            -- preserved for display
    user_text       TEXT,                            -- optional user input
    vlm_description TEXT,                            -- JSON-serialised dict
    embedding       TEXT,                            -- JSON-serialised float list
    created_at      TEXT    NOT NULL                 -- ISO-8601 UTC
);

CREATE INDEX IF NOT EXISTS idx_items_status ON items (status);
```

Source-of-truth fields: `status`, `filename`, `original_name`, `user_text`, `created_at`. **Derived fields:** `vlm_description` (produced by the VLM from the image plus `user_text`) and `embedding` (produced by the embedding model from `vlm_description.to_search_text()`). If we ever rebuild

the descriptions and embeddings from a different provider, we can drop those two columns and recompute them without losing user-supplied information.

Schema choices: why TEXT for embeddings rather than BLOB?

Two reasons: portability and inspectability. JSON text works identically across SQLite and PostgreSQL, and we did not want the production schema to depend on backend-specific vector types (PostgreSQL has `vector` via `pgvector` but it is an extension, not a default). Storing the embedding as a JSON list also means we can open the database file in any tool and read the vector by eye, which made debugging the float32 clamping bug much easier. If the embedding model ever changes dimension, the column itself stays valid – it is the comparison code that would have to reject mismatched lengths. Migrating across models is left as future work.

5.2 Caching policy

Required. What do you cache, how long, where, and how is the cache invalidated. Report cache hit rate from a demo run.

We cache exactly one thing: the result of `embed_async(text)` for the duration of a single process. The cache is a plain `dict[str, tuple[float, ...]]` keyed on the exact text string and lives at module scope in `src/services/ai_service.py`. The trade-off is simple: identical descriptions produce identical embeddings, and re-embedding the same string wastes both tokens and latency.

The cache has no TTL – entries live as long as the process does – and is reset on restart. In a long-running production process this would mean the dict grows without bound; in our tests we expose a `clear_embed_cache()` helper that fixtures call between tests to keep behaviour deterministic.

Hit rate. Running `scripts/demo.py`, which registers five lost items and five found items with matching descriptions, gives a 50 % cache hit rate (every `found` variant of a description appears second after its `lost` twin).

Correctness vs. freshness: which cached values can become stale?

The only stale-value risk is changing the embedding model mid-process. If `EMBEDDING_MODEL` flips from `text-embedding-3-small` to `text-embedding-3-large` the cache would still return the smaller vectors. Right now we do not key on model name – in production we would add `(model, text)` as the key so a configuration change naturally invalidates the relevant entries.

6. Testing

6.1 Strategy & coverage

Required. Report the coverage number. State how you mocked the AI module and the HTTP layer.

The test suite has **164 tests, all green**, and reaches **71 % line coverage** on `src/`. The provided smoke tests in `tests/test_ai_smoke.py` all pass – that is the grading contract we treat as non-negotiable.

Mocking the AI module. We never call the real provider from tests. Instead, `tests/fakes.py` defines `FakeVLM` (returns a canned `ItemDescription` and records every call into a `calls` list) and `FakeEmbedder` (returns a deterministic unit vector seeded from `hash(text)`). The service-layer wrappers accept the fake as an explicit keyword argument (`describe_item_async(..., vlm=fake_vlm)`),

so tests inject the fake without monkey-patching the provider lookup. For the few tests that exercise the wrapper itself (retries, cache hits) we `patch("src.services.ai_service.embed", ...)` to control the underlying provider call.

Mocking the HTTP layer. We use FastAPI's `TestClient`, which mounts the app under a synchronous `httpx` transport. The `get_repo` dependency is overridden via `app.dependency_overrides[get_repo] = lambda: mock_repo` so each test gets its own in-memory repository.

Suite	Coverage
`src/` (our SE layer, 19 source files)	71 %
Provided `ai/` smoke tests (`test_ai_smoke.py`)	passing (yes)

Which lines we did not cover. Two modules are deliberately under-covered: `src/cli.py` (19 %) because it is glue between argparse and the already-tested service layer (we exercise the full pipeline through `scripts/demo.py` and the e2e smoke test instead), and `src/storage/repository.py` (0 %) because the PostgreSQL backend needs a running Postgres instance and the contract tests against the shared ABC are run against SQLite. High-coverage modules are `models.py` (100 %), `config.py` (100 %), `matcher.py` (98 %), `sqlite_repo.py` (97 %), `cost_meter.py` (96 %), and `rate_limiter.py` (91 %).

Quality vs. coverage: which mocks did you have to redesign mid-project because they were testing the wrong thing?

Two redesigns we remember. First, the batch endpoint tests originally used `mock_repo.insert_item = AsyncMock(side_effect=lambda x: x)` which returned the input `Item` unchanged – with `id=None`, so `ItemResponse.from_item` silently failed for every item in the batch and the test only checked the request status code, never the response body length. We rewrote the mock with a helper that emulates the real DB by assigning monotonic IDs and `created_at` timestamps. Second, the retry test for `embed_async` originally patched the wrong symbol (`ai.embedding.embed` instead of `src.services.ai_service.embed`); we caught that one because the call count never incremented.

6.2 Notable tests

Required. Highlight at least three tests: (i) happy-path end-to-end, (ii) concurrency, (iii) error path that surfaced a real bug.

(i) Happy path – `tests/test_smoke_e2e.py::test_full_pipeline_end_to_end.` This test runs the full pipeline against an in-memory SQLite database and the offline fakes: it registers a lost item, registers a found item, asks for matches, and asserts that the top-1 candidate is the found item with a score in $[0, 1]$. If anything between the API layer and the storage layer regresses, this test fails first.

(ii) Concurrency – `tests/test_pipeline.py::test_returns_exceptions_as_values.` Verifies that when one of three concurrent tasks raises `ValueError`, `run_with_semaphore` returns the exception as a value in the result list rather than letting it propagate out of `asyncio.gather`. This is the contract our batch endpoints rely on to report partial success.

(iii) Error path that surfaced a real bug – `tests/test_api.py::test_post_batch_lost_returns_201.` This is the test that caught the mock-redesign bug described above. The test asserted `len(data) == 2`, the assertion failed with `0 == 2`, and the captured logs showed two `api.batch_register.item_failed` warnings. Following the traceback we discovered that `ItemResponse.from_item` was rejecting the mocked items because they had `id=None`; the real `SQLiteRepository.insert_item` sets it, so production was fine. We fixed the test, added a helper to emulate the DB, and the bug never made it to a release.

What didn't you test?

We did not run a real provider integration test in CI – the cost and the risk of leaking keys did not seem worth it for a coursework project. We also did not write load tests; the benchmark is a micro-benchmark, not a sustained-load study. If we had another week we would add a sustained-load test against a mocked provider with realistic latency distribution.

7. Deployment & Reproducibility

7.1 Docker image

Required. The exact `docker build` and `docker run` commands. The image size. The Python version. Multi-stage or single-stage.

```
$ docker build -t smart-lost-and-found .
$ docker run -p 8000:8000 --env-file .env smart-lost-and-found
# API at http://localhost:8000/docs
# Health check: GET /health
```

Image size: ≈ 1.02 GB. Python: 3.12-slim. The build is **multi-stage** – the builder stage installs `gcc` and `libpq-dev`, builds a virtualenv with all dependencies, and the runtime stage copies only the venv plus the application source, runs as the non-root user `appuser`, and exposes a `HEALTHCHECK` that probes `/health` every 30 seconds.

Engineering trade-offs: did you ship Alpine, slim, or full? How does the image size compare to a fresh `python:3.12-slim` (∼120 MB)?

We chose `python:3.12-slim` over Alpine because numpy, asyncpg, and several OpenTelemetry packages depend on glibc-compiled wheels; Alpine's musl libc forces those wheels to compile from source, which explodes both build time and image size. Compared to a bare `python:3.12-slim` (∼120 MB) our image is roughly 8× larger – about 200 MB of that is Streamlit and its dependencies, 150 MB is the OpenTelemetry exporter, and 100 MB is numpy plus its scientific stack. If image size mattered for this project (e.g. if we were deploying to a serverless platform with strict cold-start budgets) we would split the UI into a separate image and drop the OTel exporter for a smaller HTTP-only shipping client.

7.2 Configuration

Required. A short list of every environment variable the app reads, and what happens if each is missing.

Variable	Default	If missing or invalid
`LLM_PROVIDER`	`openai`	uses default
`LLM_MODEL`	`gpt-4o`	uses default
`OPENAI_API_KEY`	*(none)*	real provider calls fail with `ProviderError`; offline mode unaffected
`GOOGLE_API_KEY`	*(none)*	secondary failover provider unavailable
`EMBEDDING_PROVIDER`	`openai`	uses default
`EMBEDDING_MODEL`	`text-embedding-3-small`	uses default
`SECONDARY_LLM_PROVIDER`	`gemini`	failover disabled if empty
`SECONDARY_LLM_MODEL`	`gemini-2.0-flash`	secondary model identifier
`DATABASE_URL`	`sqlite+aiosqlite:///./dev.db`	uses default
`IMAGES_DIR`	`data/images`	created on first save
`MAX_IMAGE_BYTES`	`5242880`	uses default 5 MB
`SEMAPHORE_LIMIT`	`10`	uses default
`AI_CALL_TIMEOUT_S`	`30.0`	uses default
`AI_CALL_TPM_LIMIT`	`40000`	uses default
`LOG_LEVEL`	`INFO`	uses default
`ENABLE_TRACING`	`false`	tracing decorators become no-ops
`OTLP_ENDPOINT`	`http://localhost:4317`	spans go nowhere if Jaeger absent
`COST_LOG_PATH`	`artefacts/cost.jsonl`	file created on first record

8. Limitations & Future Work

Required. 3–5 bullet points. Honest list of where the system is brittle, what it would take to productionize, and what you would do differently with another week.

- **Local-only image storage.** Uploaded images are written to `data/images/` on the same machine as the API. For multi-instance production deployment we would push images to S3 or an equivalent object store and persist only a presigned URL in the database. Estimated work: two days, mostly migrations.

- **In-process embedding cache.** The cache lives in a single process and is invalidated on restart. Across multiple workers (e.g. a Gunicorn fleet) each worker re-computes embeddings independently. A shared Redis cache keyed on `(model, text)` would amortise the cost. Estimated work: one day plus an extra container in `docker-compose`.

- **No streaming for long batches.** The batch endpoints block until every item has been processed; clients with 100 images sit and wait. Server-Sent Events or WebSocket progress reporting would let the UI show per-item status. Estimated work: 3 days, mostly the UI side.

- **Cost meter is a counter, not a guard.** `CostMeter` records what we have spent but does not enforce a daily or monthly budget ceiling. Hooking a soft limit into the same `TokenBudget` that does rate limiting would refuse new calls once spend exceeds a configured dollar threshold. Estimated work: half a day.

- **PostgreSQL backend is verified by contract but not by load.** The PostgreSQL repository implements the same interface as the SQLite one and passes the contract tests against SQLite, but we have not stress tested it against a real PostgreSQL instance under concurrent writes. This is the most likely place a production issue would emerge.

Be honest: no project is perfect. What was the worst design decision you made?

The worst decision we made was binding the failover wrapper to the service-layer configuration via a free function (`build_failover_vlm_if_configured()`) rather than passing the configured provider

list as a constructor argument. It works, but it made the service layer's `describe_item_async` harder to test because tests have to either set environment variables or pass the VLM explicitly. A proper dependency injection container (or even just a factory parameter on the service module) would have been cleaner. With another week we would refactor the service layer to accept its collaborators as constructor parameters and instantiate them once in `src/api.py`'s lifespan.

9. Tools & Acknowledgements

Required. Disclose substantial AI-assistant use. For each significant LLM-aided contribution: which module, which assistant, and whether the team reviewed and adapted it.

We used **Claude Code** as a development assistant throughout the project. It is an AI coding tool from Anthropic that runs inside the terminal, reads project files directly, and produces edits the team reviewed before merging. Every line it produced was inspected by a teammate; nothing was committed unread.

- **Implementation support.** Claude Code helped scaffold the initial structure of `src/core/failover.py` (the ranked-provider iteration pattern) and contributed to the sliding-window design for `src/concurrency/rate_limiter.py`. We adjusted the constructor signatures, replaced an early `queue.Queue` sketch with a `collections.deque` guarded by an `asyncio.Lock`, and wrote the 60-second expiry logic ourselves.
- **Debugging.** When the failover wrapper was invoked *after* tenacity rather than *before*, Claude Code helped us trace the call-count assertion that surfaced the bug and proposed the nested-policy fix that we describe in §4. Similar help came on the mock-redesign issue (the batch endpoint tests returning items with `id=None`).
- **Refactoring.** It assisted with renaming, type-hint tightening (mypy clean-up across all 19 source files), and breaking large functions into smaller pieces with single responsibilities. Suggested rewrites were applied selectively after team review.
- **Documentation help.** Drafts of this report, the README, the team Git workflow plan, and the bonus-features explanation guide were AI-assisted, then edited by the team for tone, accuracy, and project-specific detail before submission.
- **Workflow organisation.** Claude Code helped plan the 33-pull-request workflow, generate consistent commit messages, and keep the PR descriptions aligned with the actual diff. Final reviews and merges were always performed by a human team member.

No production code or documentation was committed without at least one teammate reading the full diff. The AI assistant was treated as a fast, context-aware collaborator that accelerated routine work; the design decisions, judgement calls, and final wording were ours.

References

[1] OpenAI, *API reference*, https://platform.openai.com/docs/.

[2] Google, *Gemini API documentation*, https://ai.google.dev/docs.

[3] Anthropic, *Claude Code documentation*, https://docs.claude.com/en/docs/claude-code/.

[4] tenacity, *Retrying library for Python*, https://tenacity.readthedocs.io/.

[5] Encode, *FastAPI tutorial*, https://fastapi.tiangolo.com/tutorial/.

[6] Pydantic, *Settings management with pydantic-settings*, https://docs.pydantic.dev/latest/concepts/pydantic_settings/.

[7] OpenTelemetry, *Python instrumentation*, https://opentelemetry.io/docs/instrumentation/python/.

[8] Streamlit, *API reference*, https://docs.streamlit.io/library/api-reference.

[9] Docker, *Best practices for writing Dockerfiles*, https://docs.docker.com/develop/develop-images/dockerfile_best-practices/.

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20 % of the table on page 6.

```
$ git clone https://github.com/<team-org>/smart-lost-and-found.git
$ cd smart-lost-and-found
$ cp .env.example .env       # offline benchmark needs no real keys
$ python -m venv .venv
$ source .venv/bin/activate  # Windows: .venv\Scripts\activate
$ pip install -r requirements.txt
$ python scripts/bench.py --n 12
```

The script prints both runs and the speedup table to stdout. The default mode is `both`; the default semaphore is 10. For a larger load test pass `-n 24` or `-n 48`; for a different parallelism cap pass `-sem 4`.

End of report — thank you for reading.